

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Бэк-энд разработка

Отчет

Лабораторная работа №4  
«Развёртывание приложения на удалённом сервере»

Выполнила:

Олейник Полина

Группа 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

## Задача

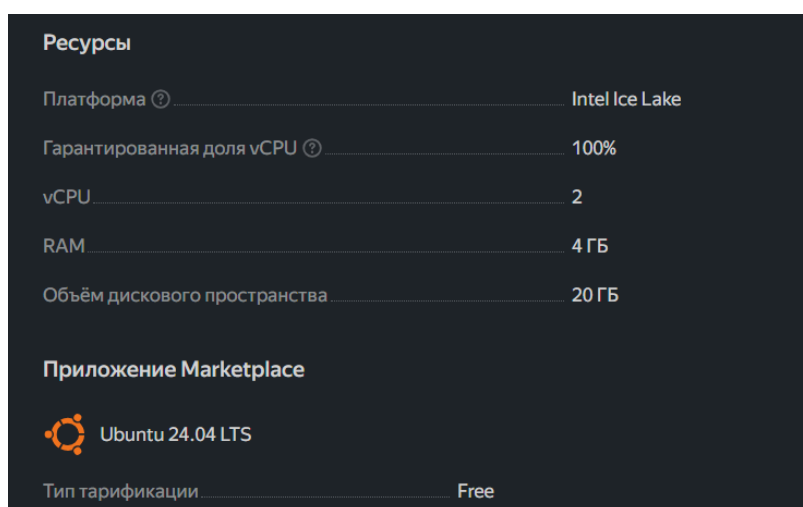
Провести предварительную настройку удалённого сервера (установка необходимых зависимостей); провести развёртывание приложения на удалённом сервере; настроить nginx в режиме обратного прокси для обеспечения доступа к приложению извне.

**Тема проекта:** вариант 1 – приложение для бронирования столиков в ресторанах. Разворачивался результат ЛР3 (докеризованные микросервисы: auth, restaurants, bookings, gateway, база данных, RabbitMQ).

## Ход работы

### 1. Подготовка сервера

Сначала я зарегистрировалась в Yandex Cloud и активировала пробный период (стартовый грант). Создала виртуальную машину со следующими параметрами: Ubuntu 24.04 LTS, платформа Intel Ice Lake, 2 vCPU со 100% гарантированной долей, 4 ГБ RAM, 20 ГБ диска.



*Параметры созданной виртуальной машины*

При создании указала SSH-ключ и разрешила входящие подключения. После того как машина перешла в состояние Running, записала её публичный IP-адрес и подключилась к ней по SSH со своего компьютера:

```
ssh -i путь_к_ключу имя_пользователя@89.169.170.37
```

## 2. Предварительная настройка сервера

Дальше на самом сервере я установила всё необходимое для развёртывания приложения. Сначала обновила систему:

```
sudo apt update && sudo apt upgrade -y
```

Поставила git — он понадобился, чтобы скопировать проект из репозитория:

```
sudo apt install -y git
```

Установила Docker и Docker Compose, так как всё приложение уже было докеризовано в ЛРЗ, и на сервере не нужно было отдельно ставить Node, PostgreSQL или RabbitMQ:

```
curl -fsSL https://get.docker.com | sudo sh  
sudo usermod -aG docker $USER
```

После этого вышла из SSH-сессии и зашла заново, чтобы группа docker применилась без sudo перед каждой командой.

Настроила брандмауэр — открыла только порты SSH и HTTP, всё остальное осталось закрытым:

```
sudo ufw allow OpenSSH  
sudo ufw allow 80/tcp  
sudo ufw enable
```

И установила сам nginx, сразу включив его автозапуск при перезагрузке сервера:

```
sudo apt install -y nginx  
sudo systemctl enable nginx
```

## 3. Развёртывание приложения

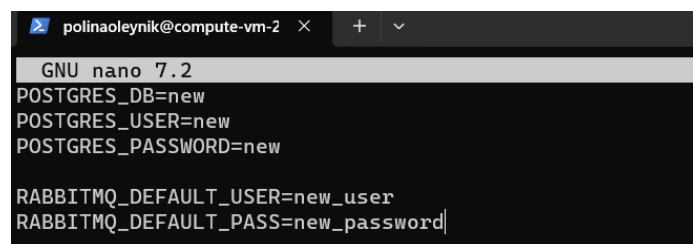
Склонирувала на сервер репозиторий со своим форком проекта и переключилась на ветку, в которой лежит докеризованное приложение из ЛРЗ:

```
git clone https://github.com/Fieria/ITMO-ACS-Backend-2026-C.git app  
cd app  
git checkout lab3  
cd "BP2.1/Олейник Полина/labs/lab3"
```

В репозитории лежат только файлы `.env.example` — сами `.env` в `git` не попадают, поэтому на сервере их нужно было создать заново, отдельно для корня проекта и для каждого сервиса:

```
cp .env.example .env
cp services/auth/.env.example services/auth/.env
cp services/restaurants/.env.example services/restaurants/.env
cp services/bookings/.env.example services/bookings/.env
cp gateway/.env.example gateway/.env
```

Дальше заполнила каждый файл настоящими значениями через `nano`, не перенося пароли и ключи со своего компьютера, а задав их заново специально для этого сервера. В корневом `.env` указала данные для контейнера с базой и RabbitMQ:

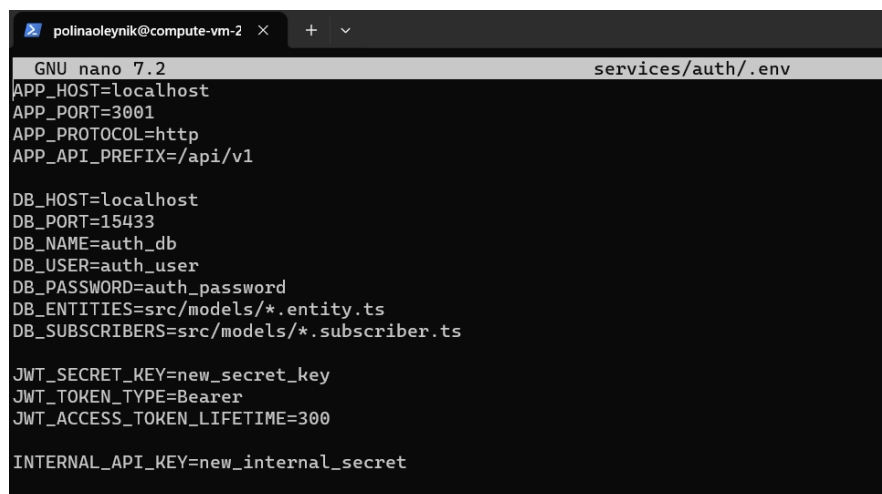
A terminal window titled 'polinaoleynik@compute-vm-2' shows the GNU nano 7.2 editor. The file being edited is the root .env. The content includes database settings: POSTGRES\_DB=new, POSTGRES\_USER=new, POSTGRES\_PASSWORD=new, and RabbitMQ settings: RABBITMQ\_DEFAULT\_USER=new\_user, RABBITMQ\_DEFAULT\_PASS=new\_password.

```
GNU nano 7.2
POSTGRES_DB=new
POSTGRES_USER=new
POSTGRES_PASSWORD=new

RABBITMQ_DEFAULT_USER=new_user
RABBITMQ_DEFAULT_PASS=new_password|
```

*Корневой .env: параметры базы данных и RabbitMQ*

В `.env` каждого сервиса — свои настройки подключения к базе, а `JWT_SECRET_KEY` и `INTERNAL_API_KEY` одинаковые во всех трёх, чтобы токены и внутренние запросы между сервисами проходили проверку:

A terminal window titled 'polinaoleynik@compute-vm-2' shows the GNU nano 7.2 editor. The file being edited is services/auth/.env. The content includes application settings (APP\_HOST, APP\_PORT, APP\_PROTOCOL, APP\_API\_PREFIX), database settings (DB\_HOST, DB\_PORT, DB\_NAME, DB\_USER, DB\_PASSWORD, DB\_ENTITIES, DB\_SUBSCRIBERS), JWT settings (JWT\_SECRET\_KEY, JWT\_TOKEN\_TYPE, JWT\_ACCESS\_TOKEN\_LIFETIME), and INTERNAL\_API\_KEY.

```
GNU nano 7.2 services/auth/.env
APP_HOST=localhost
APP_PORT=3001
APP_PROTOCOL=http
APP_API_PREFIX=/api/v1

DB_HOST=localhost
DB_PORT=15433
DB_NAME=auth_db
DB_USER=auth_user
DB_PASSWORD=auth_password
DB_ENTITIES=src/models/*.entity.ts
DB_SUBSCRIBERS=src/models/*.subscriber.ts

JWT_SECRET_KEY=new_secret_key
JWT_TOKEN_TYPE=Bearer
JWT_ACCESS_TOKEN_LIFETIME=300

INTERNAL_API_KEY=new_internal_secret
```

*services/auth/.env*

```
polinaoleynik@compute-vm-2 x + v
GNU nano 7.2 gateway/.env
APP_HOST=localhost
APP_PORT=8080
APP_API_PREFIX=/api/v1

AUTH_SERVICE_URL=http://localhost:3001
RESTAURANTS_SERVICE_URL=http://localhost:3002
BOOKINGS_SERVICE_URL=http://localhost:3003
```

*gateway/.env*

```
polinaoleynik@compute-vm-2 x + v
GNU nano 7.2 services/bookings/.env *
APP_HOST=localhost
APP_PORT=3003
APP_PROTOCOL=http
APP_API_PREFIX=/api/v1

DB_HOST=localhost
DB_PORT=15433
DB_NAME=bookings_db
DB_USER=bookings_user
DB_PASSWORD=bookings_password
DB_ENTITIES=src/models/*.entity.ts
DB_SUBSCRIBERS=src/models/*.subscriber.ts

JWT_SECRET_KEY=new_secret_key
JWT_TOKEN_TYPE=Bearer
JWT_ACCESS_TOKEN_LIFETIME=300

INTERNAL_API_KEY=new_internal_secret
RESTAURANTS_SERVICE_URL=http://localhost:3002

RABBITMQ_HOST=localhost
RABBITMQ_PORT=5673
RABBITMQ_USER=new_user
RABBITMQ_PASSWORD=new_password
```

*services/bookings/.env*

```
GNU nano 7.2 services/restaurants/.env
APP_HOST=localhost
APP_PORT=3002
APP_PROTOCOL=http
APP_API_PREFIX=/api/v1

DB_HOST=localhost
DB_PORT=15433
DB_NAME=restaurants_db
DB_USER=restaurants_user
DB_PASSWORD=restaurants_password
DB_ENTITIES=src/models/*.entity.ts
DB_SUBSCRIBERS=src/models/*.subscriber.ts

JWT_SECRET_KEY=new_secret_key
JWT_TOKEN_TYPE=Bearer
JWT_ACCESS_TOKEN_LIFETIME=300

INTERNAL_API_KEY=new_internal_secret
AUTH_SERVICE_URL=http://localhost:3001
BOOKINGS_SERVICE_URL=http://localhost:3003

RABBITMQ_HOST=localhost
RABBITMQ_PORT=5673
RABBITMQ_USER=new_user
RABBITMQ_PASSWORD=new_password
```

*services/restaurants/.env*

В `docker-compose.yml` порт `gateway` оставила сброшенным только на локальный адрес сервера, а не на весь интернет, чтобы снаружи был виден исключительно `nginx`:

```
gateway:
  ports:
    - "127.0.0.1:8080:8080"
```

После этого подняла весь стек одной командой:

```
docker compose up -d --build
```

и проверила, что все шесть контейнеров запустились и база с очередью сообщений перешли в состояние `healthy`:

```
docker compose ps
```

```
polinaoleynik@compute-vm-2-4-20-ssd-1790363084242:~/app/БР2.1/Олейник Полина/labs/lab3$ docker compose ps
```

| NAME               | PORTS                                                                                                                                                   | IMAGE                 | COMMAND                  | SERVICE     | CREATED        | STATUS                  |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|--------------------------|-------------|----------------|-------------------------|
| lab3-auth-1        | 3001/tcp                                                                                                                                                | lab3-auth             | "docker-entrypoint.s..." | auth        | 41 seconds ago | Up 21 seconds           |
| lab3-bookings-1    | 3003/tcp                                                                                                                                                | lab3-bookings         | "docker-entrypoint.s..." | bookings    | 41 seconds ago | Up 19 seconds           |
| lab3-db-1          | 0.0.0.0:15433->5432/tcp, [::]:15433->5432/tcp                                                                                                           | postgres:17           | "docker-entrypoint.s..." | db          | 56 seconds ago | Up 39 seconds (healthy) |
| lab3-gateway-1     | 127.0.0.1:8080->8080/tcp                                                                                                                                | lab3-gateway          | "docker-entrypoint.s..." | gateway     | 41 seconds ago | Up 19 seconds           |
| lab3-rabbitmq-1    | 4369/tcp, 5671/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:5673->5672/tcp, [::]:5673->5672/tcp, 0.0.0.0:15673->15672/tcp, [::]:15673->15672/tcp | rabbitmq:3-management | "docker-entrypoint.s..." | rabbitmq    | 56 seconds ago | Up 40 seconds (healthy) |
| lab3-restaurants-1 | 2003/tcp                                                                                                                                                | lab3-restaurants      | "docker-entrypoint.s..." | restaurants | 41 seconds ago | Up 20 seconds           |

Заполнила базу тестовыми данными, запустив `seed.js` не с самого сервера напрямую, а отдельным одноразовым контейнером в той же `docker`-сети – иначе скрипт не смог бы достигать до внутреннего API `auth` по имени контейнера:

```
docker compose run --rm -v "$(pwd)/scripts:/app/scripts" -e AUTH_SERVICE_URL=http://auth:3001 auth node scripts/seed.js
```

```
polinaoleynik@compute-vm-2-4-20-ssd-1790363084242:~/app/БР2.1/Олейник Полина/labs/lab3$ docker compose run --rm -v "$(pwd)/scripts:/app/scripts" -e AUTH_SERVICE_URL=http://auth:3001 auth node scripts/seed.js
```

```
[+] run 1/1
- Container lab3-db-1 Running
Container lab3-db-1 Waiting
Container lab3-db-1 Healthy
Container lab3-auth-run-6b7a02756904 Creating
Container lab3-auth-run-6b7a02756904 Created
Пользователь admin@example.com:
  зарегистрирован admin@example.com (id=1)
  роль назначена: ADMIN
Пользователь owner@example.com:
  зарегистрирован owner@example.com (id=2)
  роль назначена: RESTAURANT_ADMIN
Готово.
```

*Результат запуска `seed.js`: созданы пользователи `admin` и `owner`*

## 4. Настройка `nginx` как обратного прокси

Создала конфигурационный файл сайта:

```
sudo nano /etc/nginx/sites-available/booking-app
```

и прописала в нём единственный блок `server`, который принимает запросы на 80-м порту и перенаправляет их на gateway, слушающий 127.0.0.1:8080:

```
GNU nano 7.2 /etc/nginx/sites-available/booking-app *
server {
    listen 80;
    server_name 89.169.170.37;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

*Конфигурация /etc/nginx/sites-available/booking-app*

Включила этот конфиг, отключила конфиг по умолчанию, проверила синтаксис и перезапустила `nginx`:

```
sudo ln -s /etc/nginx/sites-available/booking-app /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

## 5. Проверка

Сначала проверила доступ к приложению снаружи – со своего компьютера, а не с самого сервера, обычным запросом на 80-й порт, без указания порта gateway:

```
curl http://89.169.170.37/api/v1/cities
```

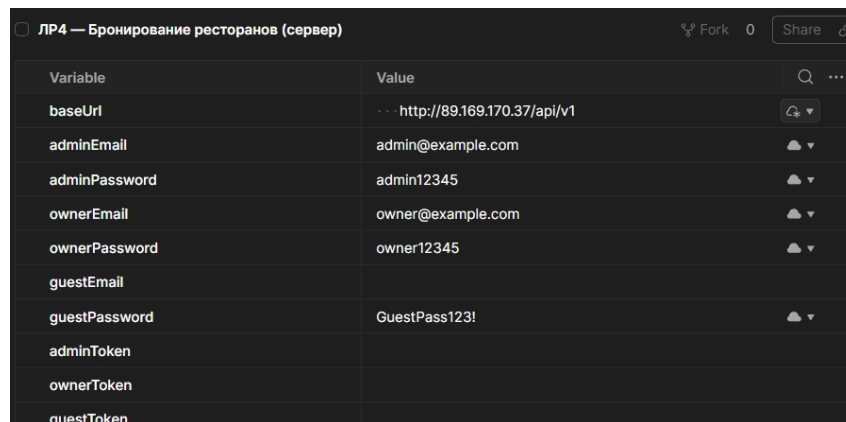
```
PS C:\Users\User> curl http://89.169.170.37/api/v1/cities

StatusCode      : 200
StatusDescription : OK
Content         : []
RawContent      : HTTP/1.1 200 OK
                  Access-Control-Allow-Origin: *
                  Connection: keep-alive
                  Keep-Alive: timeout=60
                  Proxy-Connection: keep-alive
                  Content-Length: 2
                  Content-Type: application/json; charset=utf-8
                  Date: F...
Forms           : {}
Headers         : {[Access-Control-Allow-Origin, *], [Connection, keep-alive], [Keep-Alive, timeout=60], [Proxy-Connection, keep-alive]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 2
```

*Ответ 200 OK на запрос с компьютера через nginx*

Запрос прошёл через `nginx` и вернул ответ от приложения – значит, обратный прокси настроен верно.

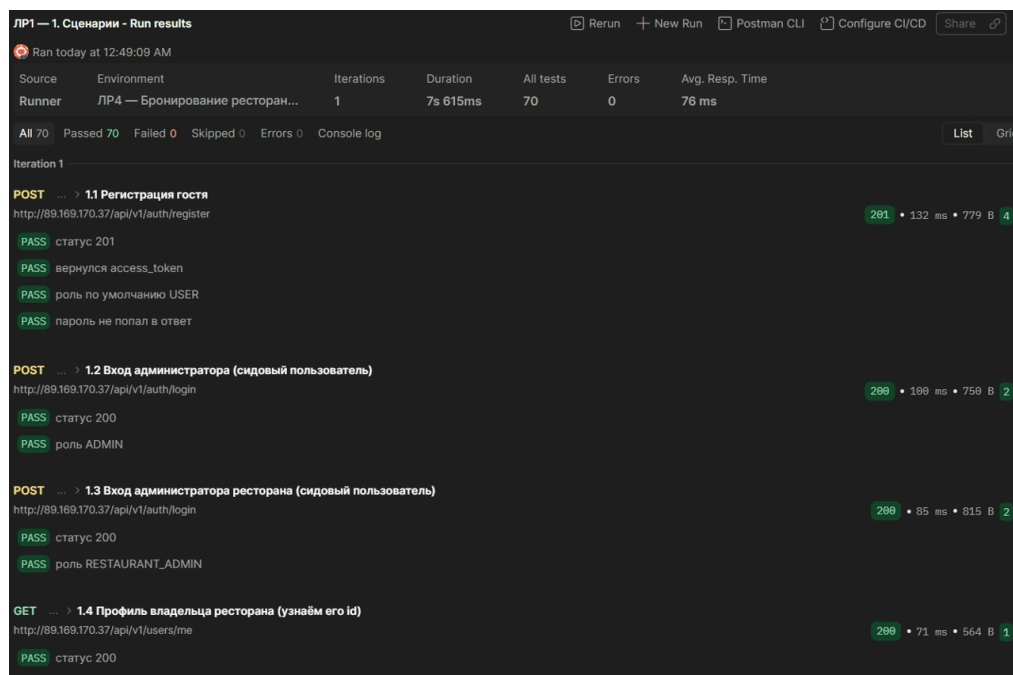
Дальше для окончательной проверки прогнала через сервер те же коллекции Postman, что использовались для ЛР2 и ЛР3. Создала отдельное окружение с адресом сервера вместо localhost:



| Variable      | Value                       |
|---------------|-----------------------------|
| baseUrl       | http://89.169.170.37/api/v1 |
| adminEmail    | admin@example.com           |
| adminPassword | admin12345                  |
| ownerEmail    | owner@example.com           |
| ownerPassword | owner12345                  |
| guestEmail    |                             |
| guestPassword | GuestPass123!               |
| adminToken    |                             |
| ownerToken    |                             |
| questToken    |                             |

*Окружение Postman «ЛР4 — Бронирование ресторанов (сервер)»*

Прогнала коллекцию «1. Сценарии» — все 70 проверок из 70 прошли успешно, без единой ошибки:



| Source | Environment                    | Iterations | Duration | All tests | Errors | Avg. Resp. Time |
|--------|--------------------------------|------------|----------|-----------|--------|-----------------|
| Runner | ЛР4 — Бронирование ресторан... | 1          | 7s 615ms | 70        | 0      | 78 ms           |

| All         | 70 | Passed | 70 | Failed | 0 | Skipped | 0 | Errors | 0 |
|-------------|----|--------|----|--------|---|---------|---|--------|---|
| Console log |    |        |    |        |   |         |   |        |   |

| Iteration                                                                                                            | 1                      |
|----------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>POST</b> ... > 1.1 Регистрация гостя<br>http://89.169.170.37/api/v1/auth/register                                 | 201 • 132 ms • 779 B 4 |
| PASS статус 201                                                                                                      |                        |
| PASS вернулся access_token                                                                                           |                        |
| PASS роль по умолчанию USER                                                                                          |                        |
| PASS пароль не попал в ответ                                                                                         |                        |
| <b>POST</b> ... > 1.2 Вход администратора (сидовый пользователь)<br>http://89.169.170.37/api/v1/auth/login           | 200 • 100 ms • 750 B 2 |
| PASS статус 200                                                                                                      |                        |
| PASS роль ADMIN                                                                                                      |                        |
| <b>POST</b> ... > 1.3 Вход администратора ресторана (сидовый пользователь)<br>http://89.169.170.37/api/v1/auth/login | 200 • 85 ms • 815 B 2  |
| PASS статус 200                                                                                                      |                        |
| PASS роль RESTAURANT_ADMIN                                                                                           |                        |
| <b>GET</b> ... > 1.4 Профиль владельца ресторана (узнаём его id)<br>http://89.169.170.37/api/v1/users/me             | 200 • 71 ms • 564 B 1  |
| PASS статус 200                                                                                                      |                        |

*Результат прогона коллекции «1. Сценарии»: 70/70, 0 ошибок*

И следом, в том же окружении, коллекцию «2. По группам орепарі» — 53 проверки из 53 тоже прошли успешно:

LP1 — 2. По группам орепарі - Run results

Ran today at 12:46:56 AM

| Source | Environment                    | Iterations | Duration | All tests | Errors | Avg. Resp. Time |
|--------|--------------------------------|------------|----------|-----------|--------|-----------------|
| Runner | LP4 — Бронирование ресторан... | 1          | 7s 63ms  | 53        | 0      | 73 ms           |

All 53 Passed 53 Failed 0 Skipped 0 Errors 0 Console log

Iteration 1

|                                                                                                           |                        |
|-----------------------------------------------------------------------------------------------------------|------------------------|
| <b>POST</b> ... > <b>Регистрация (ещё один пользователь)</b><br>http://89.169.170.37/api/v1/auth/register | 201 • 186 ms • 793 B 1 |
| <b>POST</b> Авторизация > <b>Вход</b><br>http://89.169.170.37/api/v1/auth/login                           | 200 • 81 ms • 750 B 1  |
| <b>GET</b> ... > <b>Свой профиль</b><br>http://89.169.170.37/api/v1/users/me                              | 200 • 61 ms • 564 B 1  |
| <b>PATCH</b> ... > <b>Изменить свой профиль</b><br>http://89.169.170.37/api/v1/users/me                   | 200 • 69 ms • 564 B 1  |
| <b>GET</b> ... > <b>История своих броней</b><br>http://89.169.170.37/api/v1/users/me/bookings             | 200 • 82 ms • 947 B 1  |
| <b>GET</b> ... > <b>Список городов</b><br>http://89.169.170.37/api/v1/cities                              | 200 • 64 ms • 426 B 1  |

*Результат прогона коллекции «2. По группам орепарі»: 53/53, 0 ошибок*

Оба прогона дали тот же результат, что и на локальном докеризованном стеке в ЛР3 — значит, приложение на сервере ведёт себя так же, а nginx корректно пропускает к нему весь трафик снаружи.

## Вывод

В ходе работы я развернула приложение для бронирования столиков на удалённом сервере в Yandex Cloud. Провела предварительную настройку сервера: установила git, Docker, настроила брандмауэр и поставила nginx. Склониrowала на сервер докеризованную версию приложения из ЛР3, создала для неё отдельные .env-файлы с собственными секретами, ограничила доступ к gateway только локальным адресом сервера и подняла весь стек из шести контейнеров командой `docker compose up -d`, заполнив базу тестовыми данными. Настроила nginx в режиме обратного прокси, чтобы наружу был виден только он, а не отдельные сервисы. Проверила результат: запрос с личного компьютера на 80-й порт сервера успешно дошёл до приложения через nginx, а обе коллекции Postman из ДЗ3 прошли через сервер с тем же результатом, что и на локальном запуске — 70 из 70 и 53 из 53 проверок без единой ошибки. Приложение доступно извне по адресу сервера.